APEX AND TRIGEER

What is Apex in Salesforce?

- **Apex** is a **strongly-typed, object-oriented programming language** created by Salesforce.
-
- It allows developers to write **custom business logic** on the Salesforce platform.
-
- Its **syntax is similar to Java**, but it's optimized for Salesforce's multi-tenant cloud environment.
-
- Apex is executed on **Salesforce servers** and is tightly integrated with the platform's database (objects, fields, etc.).

Key Features of Apex:

1. **Salesforce-Native** – Runs directly on Salesforce's Lightning Platform.
2. **Data-Centric** – Built-in support for **SOQL (Salesforce Object Query Language)** and **DML (Insert, Update, Delete)** operations.
3. **Multi-Tenant Aware** – Enforces **Governor Limits** to ensure fair resource usage.
4. **Event-Driven** – Can be triggered by **DML events, web service requests, or scheduled jobs**.
5. **Integrated Testing** – Has built-in testing framework (@isTest classes) for deployment.

What is a Trigger?

- A **Trigger** in Salesforce is a piece of **Apex code** that runs **automatically** when a record is **inserted, updated, deleted, or undeleted**.
- It is tied to a **specific object** (like Account, Contact, Opportunity).
- Triggers let you **customize the behavior of Salesforce's data processing** beyond what point-and-click tools (like Flows or Validation Rules) can do.

Types of Triggers:

1. **Before Triggers**
    - Run **before** a record is saved to the database.
    - Common use:
        - Validate data.
        - Set default field values.
    - Example: Setting an Account's rating before insert.
2. **After Triggers**
    - Run **after** a record has been saved in the database.
    - Common use:
        - Access system-generated fields like **Record Id**.
        - Create/update related records.
        - Call external systems.

Trigger Events

A trigger can fire on these events:

- before insert
- before update
- before delete
- after insert
- after update
- after delete
- after undelete

What are Context Variables in Salesforce?

- **Context variables** are **predefined variables** in Apex triggers that provide information about the trigger event and the records being processed.
- They are part of the Trigger class.
- They help you **identify the trigger context** (like is it before insert, after update, etc.) and access the records (Trigger.new, Trigger.old).

Context Variable	Description	Availability
Trigger.isInsert	Returns true if trigger fired due to **insert**	Before & After Insert
Trigger.isUpdate	Returns true if trigger fired due to **update**	Before & After Update
Trigger.isDelete	Returns true if trigger fired due to **delete**	Before & After Delete
Trigger.isUndelete	Returns true if trigger fired due to **undelete**	After Undelete
Trigger.isBefore	Returns true if trigger fired **before** DML	Before triggers
Trigger.isAfter	Returns true if trigger fired **after** DML	After triggers
Trigger.new	Holds **new versions** of records (can be modified in before triggers)	Insert, Update, Undelete
Trigger.old	Holds **old versions** of records (read-only)	Update, Delete
Trigger.newMap	Map of record IDs → new record values	Insert, Update, Undelete
Trigger.oldMap	Map of record IDs → old record values	Update, Delete
Trigger.size	Total number of records in trigger context	All triggers

What is Trigger.new?

- Trigger.new is a **context variable** in Apex triggers.
- It holds a **list of new versions** of the records that are being inserted or updated.
- It is **read/write** in **before triggers** (you can modify field values before saving).
- It is **read-only** in **after triggers**.

What is Trigger.old?

- Trigger.old is also a **context variable**.
- It holds a **list of old versions** of the records (the values in the database **before** the update or delete).

- It is always **read-only** (you cannot modify it).
- Not available in **before insert** (since no old record exists yet).

Differences Between Trigger.new and Trigger.old

Feature	Trigger.new	Trigger.old
Holds	New versions of records	Old versions of records
Available in	insert, update, undelete	update, delete
Modifiable?	Yes (in **before triggers**)	No (always read-only)
Usage	To access or modify new values	To compare old vs new values, or to access deleted records

Events	trigger.old	trigger.oldMap	trigger.new	trigger.newMap
before insert	✗	✗	✓	✗
after insert	✗	✗	✓	✓
before update	✓	✓	✓	✓
after update	✓	✓	✓	✓
before delete	✓	✓	✗	✗
after delete	✓	✓	✗	✗
after undelete	✗	✗	✓	✓

What are the DML statements available in Apex?

- Insert
- Update
- Delete
- Undelete
- Upsert (Combination of insert and update)
- Merge (Combination of update and delete

What is the difference between non-static and static?

- By default, all the variables and methods are non-static.
- Scope of the non-static variables or methods is within the scope of the same object.

- We can declare variables and methods as static by using static keyword.
- Scope of the static variables and methods is throughout the transaction.
- Static variables and methods, we can directly call with class name (we cannot access static variables and methods with object name).

What are the types of Collections available in Apex?

- List (ordered and allow duplicates)
- Set (unordered and won't allow duplicates)
- Map (Key and value pair)

List:

- An Ordered collection of elements.
- Each element of list has an index for identification.
- Index position of first element is always 0.
- List can be nested and even multidimensional.
- Elements can be of any data type:
    - Primitive types, collections, sObjects,
    - User-Defined Types, Built-in Types

List example:

- List myList = new List();
- List accList = new List();
- List myList = new List(); myList.add(20); Integer i = myList.get(0); myList.add(1, 30);
- List accList = [SELECT ID, Name FROM Account];

Set:

- An Unordered collection of elements.
- It doesn’t contain duplicate elements.
- Sets can contain collections that can be nested within one another.
- Elements can be of any data type:
    - Primitive types, collections, sObjects,
    - User-Defined Types, Built-in Types.

Set example:

Set stringSet = new Set();

```
Set accIdSet= new Set();

Set intSet = new Set();
intSet.add(20);
intSet.add(30);
intSet.remove(20);
Boolean b = intSet.contains(30);
Integer s = intSet.size( );
Boolean b = intSet.isEmpty( )
```

Map:

- A Map is a collection of key - value pairs.
- Keys are always unique having a value associated.
- Values can be duplicate.
- Adding a map entry with an existing key overrides the existing entry with new.
- Map key and values can contain any collection and can contain nested collections.
- Keys and values can be of any data type:
    - Primitive types, collections, sObjects,
    - User-Defined Types, Built-in Types.

Map example:

1. Map strToStrMap = new Map ();
2. Map intToStrMap = new Map();
 i. intToStrMap.put(1, 'First');
ii. intToStrMap.put(2, 'Second');
iii. Boolean b = intToStrMap.containsKey(1);
 iv. String value = intToStrMap.get(2);
 v. Set s = intToStrMap.keySet();
3. Map IdToAccountMap = New Map();
4. List accList = [SELECT ID, Name FROM Account];
 5. Map IdToAccountMap = New Map(accList); 6. Map> accToConMap = new Map>( );

What are the map methods available in Apex?

- keyset (): To fetch only keys from the map.
- values (): To fetch only values from the map.
- contains Key(value): To search a key from the map.
- get(key): By supplying the key we can fetch the value.
- put(key, value): To add key and value in a map.

What is the difference between List and Set?

List	Set
List is Ordered.	Set is unordered.
List allows duplicates.	Set doesn't allow duplicates.
We can access list elements with index.	Set elements cannot be accessed with index.
We can sort list elements with sort method (default sorting order is ascending).	sort method is not available for Set.
Contains method is not available in List.	Contains method is available for Set to search for a particular element in the set.
We can process records which are stored in list using DML statements (insert, update, delete and undelete).	We cannot process records which are stored in set using DML statements.

What is the maximum size of the list?

No limit for the size of a list. It only depends on the heap size which is 6 MB (Synchronous) and 12 MB (Asynchronous).

SOQL (Salesforce Object Query Language)

SOQL is like **SQL for Salesforce**.

- Used to **query records** from one or more Salesforce objects.
- Returns data in the form of **sObjects** (Account, Contact, etc.).
- Can query **standard** and **custom objects**.

Syntax:

```
SELECT fieldList
FROM ObjectName
WHERE conditions
ORDER BY fieldName
LIMIT n
```

Example:

```
List<Account> accList = [SELECT Id, Name, Industry
                         FROM Account
                         WHERE Industry = 'Banking'
                         ORDER BY Name
                         LIMIT 10];
```

This gets the **first 10 Accounts in Banking industry**.

SOSL (Salesforce Object Search Language)

SOSL is used to **search text across multiple objects** at once.

- Works like a **global search**.
- Can search in multiple objects & fields at the same time.
- Returns data as a **list of lists of sObjects** (since multiple objects may be queried).

Syntax:

```
FIND 'searchString'
IN ALL FIELDS
RETURNING Object1 (Field1, Field2), Object2(Field1)
```

Example:

```
List<List<SObject>> searchResults =
   [FIND 'Acme'
    IN ALL FIELDS
    RETURNING Account(Id, Name), Contact(Id, Name, Email)];
```

This searches for the word **"Acme"** in **all fields** of Account & Contact,
and returns Accounts + Contacts in a list of lists.

Key Differences Between SOQL and SOSL

Feature	SOQL	SOSL
Full form	Salesforce Object Query Language	Salesforce Object Search Language
Purpose	Query records from **one object (or related)**	Search text across **multiple objects**
Search type	Structured query (specific fields/conditions)	Free-text search (keywords)

Feature	SOQL	SOSL
Return type	List<sObject>	List<List<sObject>>
Example use case	Get all Accounts in "Banking" industry	Find "Acme" in Accounts, Contacts, Leads

---

Bulkification:

- **Bulkification** means writing Apex code so it can handle **multiple records at once** in a **single transaction**.
- Salesforce executes triggers, classes, and processes in **bulk mode** (e.g., when data is loaded via Data Loader, API, batch jobs, etc.).
- If your code is written only for a **single record**, it will fail or hit **governor limits** when multiple records are processed.

In short: **Bulkification = making your code governor-limit safe and bulk-safe.**

Why is Bulkification Needed?

- Salesforce is a **multi-tenant platform** → resources are shared.
- To prevent a single org from consuming too many resources, Salesforce enforces **Governor Limits** (e.g., 150 DML statements, 100 SOQL queries per transaction).
- Without bulkification, you may run **one DML per record**, which quickly exceeds these limits.

How to Bulkify Your Code?

1. **Avoid DML or SOQL inside loops.**
    - Collect data in Lists/Sets/Maps, then perform DML once.
2. **Use Collections (List, Set, Map).**
    - Work on groups of records, not single ones.
3. **Leverage Maps for Parent-Child relationships.**
    - Efficiently link related data.
4. **Use Trigger.new and Trigger.old properly.**
    - Handle all records in trigger context, not just Trigger.new[0].
5. **Test with bulk data (200 records).**
    - Always write test classes that insert/update **200 records at once**.

Difference between insert/update and Database. Insert/ Database.update?

insert/update	Database. Insert/Database.update

Assume that you are inserting 100 records. If any one of the records fail due to error then entire operation will fail. None of the records will be saved into the database.	Assume that you are inserting 100 records. If any one of the records fail due to error then it will perform partial operation (valid records will be inserted/updated) if we use Database. Insert(list, false)/ Database. Update(list, false).
With insert/update if we use try-catch then we can capture only one error which will cause to stop the operation.	With Database. Insert/Database.update we can capture all the errors by saving result in Database.saveResult[].

What is Governor Limit?

● As apex runs in a multitenant environment so Apex runtime engine strictly enforces limits.
● This is because runaway Apex code or processes don't monopolize shared resources.
● If some apex code exceeds a limit then associated governor issues a runtime exception that cannot be handled.
● These limits count for each Apex transaction.
● For Batch Apex, these limits are reset for each execution of a batch of records in the execute method.

Governor Limits in Apex :

Limit Type	**Value**
SOQL queries per transaction	**100**
SOSL queries per transaction	**20**
DML statements per transaction	**150**
Records retrieved by SOQL query	**50,000**
Records processed by DML	**10,000**
Heap size	**6 MB (sync), 12 MB (async)**
CPU time per transaction	**10,000 ms (sync), 60,000 ms (async)**
Callouts (HTTP/Web services) per transaction	**100**
Future/Queueable jobs	**50 enqueued**

What is Sharing in Apex?

- In Salesforce, **sharing rules** and **org-wide defaults (OWD)** control which records a user can see.
- By default, Apex **runs in system mode** → it ignores sharing rules (so the user may see more data than expected).
- To control this, we use **with sharing**, **without sharing**, and **inherited sharing** keywords in Apex classes.

1. **with sharing**

- Enforces **sharing rules** of the current user.

- User only sees the records they have access to.

Example:

```
public with sharing class AccountService {

   public static List<Account> getUserAccounts(){

      return [SELECT Id, Name FROM Account];
   }
}
```

If OWD = **Private**, and the user has access to only 5 Accounts, they will get **5 records only**.

2. **without sharing:**

- **Ignores sharing rules** and runs in **system mode**.
- User can access **all records**, even if they don't have sharing access.

Example:

```
public without sharing class AdminService {

   public static List<Account> getAllAccounts(){

      return [SELECT Id, Name FROM Account];
   }
}
```

Even if OWD = Private, the user may see **all Accounts**.

Use carefully — only for admin/system processes.

3. inherited sharing:

- Introduced in **Spring '18 Release**.
- Class **inherits the sharing setting** from the class/method that calls it.
- If caller is with sharing → it enforces sharing.
- If caller is without sharing → it ignores sharing.

Example:

```
public inherited sharing class ContactService {

   public static List<Contact> getContacts(){

      return [SELECT Id, Name, Email FROM Contact];

   }

}
```

Behaviour depends on **who calls it**:

- If called from a ***with sharing*** class → applies sharing rules.
- If called from a ***without sharing*** class → runs in system mode.

Best Practices

- Use **with sharing** for most user-facing classes (respect data security).
- Use **without sharing** only for admin/system logic (e.g., batch jobs, triggers).
- Use **inherited sharing** for **reusable utility/service classes**, so behaviour depends on the caller.

What is Asynchronous Apex?

- **Asynchronous Apex** means code that runs **in the background**, separately from the main user transaction.
- Normal (synchronous) Apex: User clicks → Code runs immediately → User waits for it to finish.
- Asynchronous Apex: User clicks → Code continues running in the background → User doesn't have to wait.

This is essential for **long-running, heavy, or delayed jobs** (like batch processing, callouts, or scheduled tasks).

Types of Asynchronous Apex Methods:

1. Future Methods
2. Batch Apex
3. Schedule Apex
4. Queueable method

What is a Future Method?

- A **future method** is an Apex method annotated with @future.
- It runs **asynchronously in the background**, after the current transaction finishes.
- Useful for tasks that don't need to finish right away (like callouts, sending emails, updates to related records).

Think of it as **"Do this later in the background"**.

Key Features of Future Methods

1. Defined using the **@future annotation**.
2. Must be **static** and return **void**.
3. Accepts only **primitive data types**, collections of primitives, or Ids as parameters (not sObjects directly).
4. Can optionally allow **callouts** using @future(callout=true).
5. Cannot call another **future method** (no nesting).
6. Governor limit: **50 future method calls per transaction**.

Syntax:

```
@future
public static void methodName(DataType parameter) {

   // async logic here
}
```

Use Cases for Future Methods

- **Making callouts** to external services after DML.
- **Sending emails** or notifications in the background.
- **Updating related records** without slowing down the main process.
- **Executing non-critical operations** after transaction commit.

Limitations

- Cannot pass sObjects as parameters.
- Cannot return values (always void).
- Cannot be called from another future method, batch, or queueable job.
- Limited to **50 calls per transaction**.

Batch Apex

- **Batch Apex** is used to process **large volumes of records** (even millions) in manageable chunks (called **batches**).
- Normal Apex has a **limit of 50,000 records per SOQL query**, but Batch Apex can handle **millions of records** because it runs asynchronously in smaller chunks.
- Each batch runs as a **separate transaction**, so governor limits apply to each chunk individually.

Think of Batch Apex as **"breaking a big job into smaller jobs"**.

How Batch Apex Works?

1. You write a class that implements **Database.Batchable<SObject>**.
2. The class has **three methods**:
    - **start** → Defines the scope of records (query).
    - **execute** → Processes each batch of records.
    - **finish** → Runs after all batches are done (for final actions like notifications).
3. You run it with **Database.executeBatch()**.

Structure of Batch Apex:

```
global class BatchExample implements Database.Batchable<SObject> {

    // Step 1: Start method - defines records to process
    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator('SELECT Id, Name FROM Account');
    }

    // Step 2: Execute method - processes each batch (default = 200 records per batch)
    global void execute(Database.BatchableContext bc, List<Account> accList) {
        for(Account acc : accList) {
            acc.Rating = 'Warm';
        }
        update accList;
    }

    // Step 3: Finish method - runs after all batches are complete
    global void finish(Database.BatchableContext bc) {
        System.debug('✅ Batch Completed Successfully!');
    }
}
```

To execute:

```
Database.executeBatch(new BatchExample(), 200);
```

(Here 200 = batch size, i.e., process 200 records per execution).

Use Cases for Batch Apex

- **Data Cleanup** → Update millions of records (e.g., fixing old field values).
- **Reprocessing** → Recalculate rollups, sync data with another system.
- **Archiving Data** → Move old records to another object.
- **Large Integrations** → Push large datasets to external systems.

Governor Limit Benefits

- Each batch runs in its **own transaction**.
- So if you process 1 million records in 5000 batches of 200:
    - Each batch gets **its own governor limits**.
    - If one batch fails, others still succeed.

Scheduled Apex

- **Scheduled Apex** lets you run Apex code **at a specific time** or on a **recurring schedule** (daily, weekly, monthly, etc.).
- Think of it like a **cron job in Salesforce**.
- Runs **asynchronously in the background**.

Example: Run a job every night at 2 AM to clean up old records.

How to Create a Scheduled Apex Class?

1. The class must **implement the `Schedulable` interface**.
2. You must define the **`execute()` method** (this is where your logic goes).
3. Schedule it using:
    - **Setup → Apex Classes → Schedule Apex** (UI way)
    - Or using **`System.schedule()`** in code.

Syntax

```
global class MyScheduledClass implements Schedulable {
   global void execute(SchedulableContext sc) {

      // Your logic here
      System.debug('Scheduled job is running...');
   }
}
```

Scheduling the Job

From Developer Console (Execute Anonymous):

```
String cron = '0 0 2 * * ?';  // Run daily at 2 AM
System.schedule('Daily Job', cron, new MyScheduledClass());
```

Cron Expression Format

A Salesforce cron expression has **7 fields**:

Seconds Minutes Hours Day_of_month Month Day_of_week Year (optional)

Examples:

- "0 0 2 * * ?" → Every day at 2 AM
- "0 0 12 ? * MON-FRI" → Every weekday at 12 PM
- "0 0 22 L * ?" → Last day of every month at 10 PM

Use Cases of Scheduled Apex

- Nightly data cleanup (archive old records).
- Daily/weekly email reminders.
- Weekly recalculation of fields or statistics.
- Sync Salesforce data with external systems at fixed times.

Limitations

- Only **100 scheduled jobs** can be active at once.
- Scheduled jobs **run in system context** (ignores sharing unless class has with sharing).
- Long-running jobs should be written in **Batch Apex**, and scheduled jobs can trigger batch jobs.

Queueable Apex

- Salesforce introduced **Queueable Apex** as a middle ground between **Future methods** (simple but limited) and **Batch Apex** (powerful but heavy).
- It is best when you need **async jobs with more control** than future, but don't want the overhead of batch.

How Queueable Works?

- You define a class that **implements Queueable**.
- Salesforce places it in an **async job queue**.
- A **system thread** picks it up later and runs the logic.
- Each job runs in its own **execution context with fresh governor limits**.

Structure of a Queueable Class

```
public class MyQueueableJob implements Queueable {
   public void execute(QueueableContext context) {

      // Your async code here
   }
}
```

Enqueueing a Job

```
Id jobId = System.enqueueJob(new MyQueueableJob());
System.debug('Job Id: ' + jobId);
```

☞ You can check job status in **Setup → Apex Jobs** or with SOQL:

SELECT Id, Status, JobType FROM AsyncApexJob WHERE Id = :jobId

Advantages of Queueable Apex

1. **Pass Complex Data** – Unlike future methods, you can pass **sObjects, Lists, Maps, Wrappers**.
2. **Job Chaining** – You can start another Queueable job inside execute().
3. **Job Monitoring** – You get a Job ID (AsyncApexJob) to track progress.
4. **Governor Limits Reset** – Each job runs in its own transaction.

Governor Limits for Queueable

- **Max 50 jobs** enqueued per transaction.
- One Queueable job can **chain only one child job** directly (but that child can enqueue another).
- Runs in async context, so governor limits (SOQL 50,000 rows, DML 10,000, etc.) apply per job execution.

Queueable vs Future vs Batch Apex

Feature	Future	Queueable	Batch Apex
Async execution	✅ Yes	✅ Yes	✅ Yes
Pass complex objects	❌ No	✅ Yes	✅ Yes
Chaining jobs	❌ No	✅ Yes	✅ Yes
Job Monitoring	❌ No	✅ Yes	✅ Yes
Handles millions	❌ No	❌ No (medium)	✅ Yes
Best use case	Simple async task	Medium async task with chaining	Heavy processing of millions of records

---

Apex Best Practices

1. **Bulkify Code**
   - Always write code to handle multiple records (SOQL/DML outside loops).
2. **One Trigger per Object**
   - Keep a single trigger per object, and move logic to a trigger handler class for clarity.
3. **Avoid Hardcoding IDs/Values**
   - Don’t hardcode record type IDs, profile IDs, or picklist values. Use **Custom Metadata, Settings, or Schema methods**.
4. **Use Collections Efficiently**
   - Use **Set** for unique values, **Map** for fast lookups, and **List** for ordered records.
5. **Minimize SOQL & DML Statements**
   - Combine queries and DML into as few operations as possible. Never put them inside loops.
6. **Respect Governor Limits**
   - Be mindful of Apex limits (e.g., max 100 SOQL queries, 150 DML per transaction). Optimize code accordingly.
7. **Use Asynchronous Apex Wisely**
   - Use **Future, Queueable, Batch, Scheduled** for heavy processing, callouts, or large data jobs.
8. **Exception Handling**
   - Wrap risky operations in **try-catch** blocks and create **custom exceptions** for better debugging.
9. **Testing Best Practices**
   - Write tests for positive, negative, bulk, and async scenarios. Use **Test.startTest() / Test.stopTest()**.

10. **Trigger Context Variables**
    - Use **Trigger.isInsert, Trigger.isUpdate**, etc., and prevent recursion with static flags.
11. **Naming Conventions**
    - Use meaningful, consistent names for classes, variables, and methods for readability.
12. **Reusable & Modular Code**
    - Break large logic into **utility/helper classes** to avoid duplication.
13. **Use Custom Labels**
    - Store text (like error messages) in **Custom Labels** for easy localization and updates.
14. **Sharing & Security**
    - Use **with sharing** or **without sharing** properly. Enforce FLS and CRUD checks in Apex.
15. **Query Only Required Fields**
    - Avoid **SELECT ***. Query only necessary fields to reduce heap size and improve performance.

Best Practices for Test Classes

1. **Minimum 75% Code Coverage**
    - Salesforce requires at least **75% coverage** for deployment, but aim for **90–100% with meaningful tests**.
2. **Test Positive & Negative Scenarios**
    - Cover **happy paths** (valid data) and **failure cases** (invalid data, exceptions).
3. **Test Bulk Records**
    - Always test with **200+ records** to simulate real trigger bulk execution.
4. **Use Test.startTest() and Test.stopTest()**
    - Ensures async jobs (Future, Queueable, Batch, Schedule) run during the test.
    - Resets governor limits for accurate testing.
5. **Create Your Own Test Data**
    - Do **not** depend on existing org data. Use **@testSetup** methods to insert reusable test data.
6. **Use @isTest Annotation**
    - Mark test classes and methods with @isTest. It ensures they don't count against org limits.
7. **One Assert per Logical Test**
    - Use System.assert() to validate expected results, not just for code coverage.
8. **Avoid Hardcoding IDs**
    - Never hardcode record IDs in tests. Instead, create records dynamically in the test itself.
9. **Test Asynchronous Code**
    - For **Future, Queueable, Batch, Scheduled** jobs, always wrap in Test.startTest() / Test.stopTest().
10. **Use SeeAllData=false (default)**
    - Prevent dependency on org data. Create your own test data.
11. **Isolate Tests**
    - Each test should be independent and not rely on the result of another.
12. **Test Security (CRUD & FLS)**
    - Test whether your code behaves correctly when users have restricted field/object access.
13. **Test Governor Limits (Optional)**
    - Use **Limits.getQueries(), Limits.getDMLStatements()** to check your code is efficient.
14. **Test Exception Handling**
    - Validate that exceptions are thrown when expected using **System.assertException()**.
15. **Naming Conventions**
    - Use clear names like **testInsertAccount_Positive(), testInsertAccount_Negative()**.

Recursive Trigger

- A **recursive trigger** occurs when a trigger keeps calling itself **indirectly or directly**, causing an **infinite loop**.
- This usually happens when a trigger **updates the same object it's running on**.

Example:

- Trigger on **Account** updates related **Contacts**.
- Updating Contacts causes a trigger on Contact to update Account again.
- This leads to **recursion** → trigger fires repeatedly until limits are hit.

Problem with Recursive Triggers

- May hit **Governor Limits** (like SOQL, DML).
- May cause **infinite loops** and errors.
- Bad for performance and data integrity.

Preventing recursive triggers:

- Use **static Boolean variable** in a helper class.
- Check if **field values actually changed** before updating.
- Maintain a **Set<Id> of processed records** to avoid re-processing.
- Avoid unnecessary **DML operations** inside triggers.
- Use a **Trigger Framework** (like fflib Apex Common).
- Ensure **one trigger per object** with proper handler logic.

Wrapper Class

- A **custom Apex class** that wraps (groups) **multiple data types or objects** into a **single object**.
- Think of it like a **container** to hold different values (sObjects, primitives, collections, etc.) together.

Why Do We Use Wrapper Classes?

- To combine **different objects or data types**.
- To add **extra fields** (like a checkbox) along with sObject records.
- To **transfer complex data** between Apex and Visualforce/LWC.
- To build **custom data structures** not directly available in Salesforce.

Example Wrapper Class

```
public class AccountWrapper {
    public Account acc {get; set;}
    public Boolean isSelected {get; set;}

    public AccountWrapper(Account a, Boolean selected) {
        this.acc = a;
        this.isSelected = selected;
    }
}
```

Usage:

```
List<AccountWrapper> wrapperList = new List<AccountWrapper>();
for(Account a : [SELECT Id, Name FROM Account LIMIT 5]) {
   wrapperList.add(new AccountWrapper(a, false));
}
```

Here, each wrapper holds **Account record + a checkbox flag**.

Use Cases of Wrapper Class

- Show records with **checkboxes** on Visualforce/LWC.
- Combine data from **multiple objects** into one class.
- Create **custom JSON response** for REST APIs.
- Add **extra info** (flags, counts, status) with standard objects.
